COURSE NAME: ***SOFTWARE ENGINEERING AND PROJECT MANAGEMENT***

COURSE CODE: BCS501

Module 3 - Agile Development and Principles that Guide Practice

---

3.1 What is Agility?

Agility is the ability of a software team to respond effectively to change. Change is inevitable in software projects – the customer's needs shift, the market moves, new technology arrives, and team members come and go. An agile approach treats this change as normal and builds the process around it instead of resisting it.

Agility is more than a quick response. In the Pressman/Jacobson sense it covers the whole way a team works:

- Effective (rapid and adaptive) response to change.
- Effective communication among all stakeholders – developers, customers and managers.
- Drawing the customer onto the team so that they help steer the project.
- Organising the team so that it is in control of the work it performs.
- Rapid, incremental delivery of working software – small increments rather than one big release.
- A focus on people and their interaction, rather than on process, tools and heavy documentation.

An **agile team** is a nimble team able to appropriately respond to changes. Agility is therefore a philosophy, not a single method; agile process models such as XP and Scrum are concrete ways of putting that philosophy into practice.

3.2 Agility and the Cost of Change

In conventional software engineering, the cost of making a change rises **non-linearly** as the project moves along. A change requested during requirements gathering is cheap (a pencil-and-paper edit). The same change during design means reworking the architecture; during coding it means altering code; during testing it means altering code and re-testing; after delivery it can cost 60–100 times as much. This belief shaped the traditional rule: "get the requirements right before you design or code."

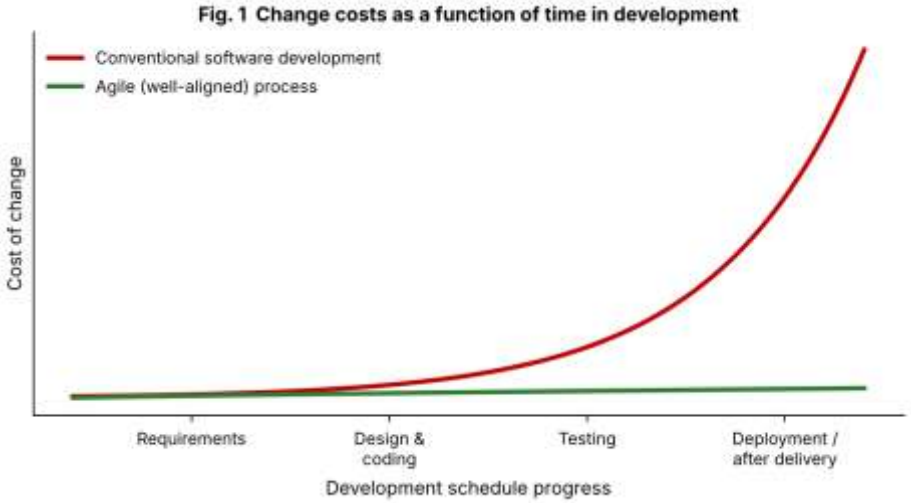


Fig. 1 Change costs as a function of time in development

Proponents of agility argue that a well-designed agile process **flattens the cost-of-change curve**. Practices such as **continuous integration, test-driven development, refactoring and small incremental releases** allow a team to accommodate change late in the project without the cost and time impact rising dramatically. The claim is debated, but the aim is clear: changes should stay cheap throughout the project.

3.3 What is an Agile Process?

Any agile software process is characterised by the way it addresses three key assumptions about most software projects:

1. It is difficult to **predict in advance** which software requirements will persist and which will change, and equally difficult to predict how customer priorities will change.
2. For many types of software, **design and construction are interleaved** – both should be performed together so that design models are proven as they are created. It is hard to predict how much design is necessary before construction is used to test the design.
3. **Analysis, design, construction and testing are not as predictable** (from a planning point of view) as we might like.

To manage this unpredictability, an agile process must be **adaptable**. Continual adaptation without forward progress accomplishes little, so an agile software process must adapt **incrementally**. To achieve incremental adaptation, an agile team requires **customer feedback** so that the appropriate adaptations can be made. An effective catalyst for this feedback is an operational prototype or a portion of an operational system delivered in short **increments**.

The Agile Manifesto

In 2001, Kent Beck and sixteen other leading developers, writers and consultants (the "Agile Alliance") signed the **Manifesto for Agile Software Development**. It states that we are uncovering better ways of

developing software and have come to value:

We value more…	…than
Individuals and interactions	processes and tools
Working software	comprehensive documentation
Customer collaboration	contract negotiation
Responding to change	following a plan

The items on the right are not worthless – the manifesto simply values the items on the left more.

The Twelve Agile Principles

The Agile Alliance defined twelve principles for those who want to achieve agility:

1. Our highest priority is to **satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference for the shorter time scale.
4. Business people and developers must **work together daily** throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information is **face-to-face conversation**.
7. **Working software is the primary measure of progress.**
8. Agile processes promote **sustainable development**. Sponsors, developers and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to **technical excellence and good design** enhances agility.
10. **Simplicity** – the art of maximising the amount of work not done – is essential.
11. The best architectures, requirements and designs emerge from **self-organising teams**.
12. At regular intervals the team **reflects on how to become more effective**, then tunes and adjusts its behaviour accordingly.

Human Factors

Agile development focuses on the talents and skills of individuals, moulding the process to specific people and teams. The following traits must exist among the people on an agile team and the team itself:

Trait	Meaning
Competence	Innate talent, specific software-related skills and overall knowledge of the process the team has chosen.
Common focus	Although members perform different tasks, all focus on one goal – delivering a working software increment to the customer within the time promised.
Collaboration	Team members collaborate with each other and with all other stakeholders to create, gather and assimilate information.
Decision-making ability	The team is given autonomy – control over both technical and project decisions.
Fuzzy problem-solving ability	The team accepts that it will continually deal with ambiguity and change; today's problem may not be tomorrow's.
Mutual trust and respect	The team becomes a "jelled" team that is stronger than the sum of its parts.
Self-organisation	The team organises itself for the work, the process to best its local environment, and the work schedule to best achieve delivery.

3.4 Extreme Programming (XP)

Extreme Programming is the most widely used approach to agile software development. Its roots go back to the late 1980s; the foundation work was done by **Kent Beck** (and Ward Cunningham) and described in Extreme Programming Explained (1999). XP uses an **object-oriented** approach as its preferred development paradigm and encompasses a set of rules and practices within the context of four framework activities: **planning, design, coding and testing**.

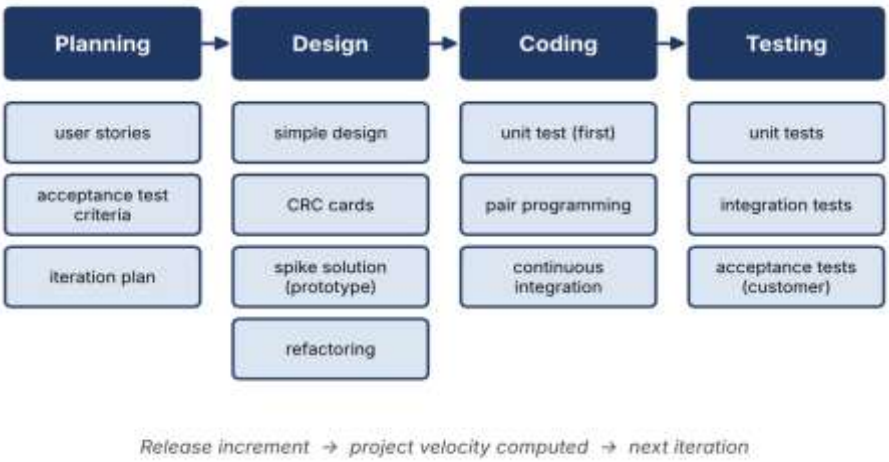
XP Values

Beck defines a set of five values that establish a foundation for all work performed as part of XP: **communication, simplicity, feedback, courage and respect.** Each value is used as a driver for specific XP activities, actions and tasks.

- **Communication** – achieved through close, informal collaboration with the customer, simple designs, CRC cards and frequent verbal contact; documentation is kept minimal.
- **Simplicity** – design only for today's needs (the "KISS" mindset), not for tomorrow's speculative ones.
- **Feedback** – comes from the implemented software itself (unit tests), from the customer (acceptance tests) and from the team (planning).
- **Courage (discipline)** – the discipline to design for today, and the courage to refactor or throw away code that is no longer good.

- **Respect** – respect among team members, for other stakeholders and for the software itself.

Fig. 2 The Extreme Programming (XP) process



The XP Process – Four Framework Activities

1. Planning. Planning begins with listening – a requirements-gathering activity that enables the technical members of the XP team to understand the business context and get a broad feel for the required output and features.

- Listening leads to the creation of a set of **"stories" (user stories)** that describe required output, features and functionality. Each story is written by the customer and placed on an **index card**.
- The customer assigns a **value (priority)** to the story based on the overall business value of the feature or function.
- XP team members assess each story and assign a **cost**, measured in development weeks. If a story will require more than three development weeks, the customer is asked to split the story.
- Customers and the XP team together decide how to group stories into the next release (the **next software increment**). Once a basic commitment is made, the team orders the stories for development in one of three ways: all stories immediately, highest-value stories first, or riskiest stories first.
- After the first release is delivered, the XP team computes **project velocity** – the number of customer stories implemented during the first release. Velocity is then used to estimate delivery dates and schedule for later releases, and to decide whether over-commitment has occurred.

2. Design. XP design rigorously follows the **KISS (Keep It Simple, Stupid)** principle. A simple design is always preferred over a more complex representation, and the design provides implementation guidance for a story as written – nothing more, nothing less.

- XP encourages the use of **CRC (Class-Responsibility-Collaborator) cards** as an effective mechanism for thinking about the software in an object-oriented context. CRC cards identify and organise the classes relevant to the current increment.
- If a difficult design problem is encountered, XP recommends the immediate creation of an operational prototype of that portion of the design, called a **spike solution**, to lower risk and validate the original estimates.
- XP encourages **refactoring** – a construction technique that is also a design technique. Refactoring is the process of changing a software system in such a way that the external behaviour of the code does not change yet the internal structure is improved. Design is therefore a continuous activity rather than a one-time phase.

3. Coding. XP recommends that after stories are developed and preliminary design work is done, the team should **not** move to code, but rather develop a series of **unit tests** that will exercise each of the stories to be included in the current release.

- Once the unit test has been created, the developer is better able to focus on what must be implemented to pass it. Once the code is complete, it can be unit-tested immediately, providing instantaneous feedback.
- A key concept during the coding activity is **pair programming**: two people work together at one computer workstation to create code for a story. This provides a mechanism for real-time problem solving and real-time quality assurance, and keeps the developers focused on the problem. Members of the pair take slightly different roles – one thinks about the coding detail, the other ensures that the standards are followed and the code fits the story.
- As pair programmers complete their work, the code they develop is integrated with the work of others. This **continuous integration** strategy helps to avoid compatibility and interfacing problems and provides a "smoke-testing" environment that helps to uncover errors early.

4. Testing. The unit tests created before coding are implemented using a framework that enables them to be automated, so they can be executed easily and repeatedly. This encourages a **regression testing** strategy whenever code is modified.

- **Integration, validation and system testing** can be done on a daily basis, giving the XP team a continual indication of progress and early warning of trouble.
- **Acceptance tests (customer tests)** are specified by the customer and focus on overall system features and functionality that are visible and reviewable by the customer. They are derived from the user stories that have been implemented as part of a software release.

Industrial XP (IXP)

Joshua Kerievsky describes **Industrial Extreme Programming (IXP)** as an organic evolution of XP. It

is customer-centric, with a greater degree of "formality", and it targets large organisations. IXP incorporates six new practices:

- **Readiness assessment** – checks that an appropriate development environment exists, the team is composed of the proper stakeholders, the organisation has a distinct quality programme and supports continuous improvement, and the community supports the new culture.
- **Project community** – the "team" is replaced by a community of people with the right skills and training, including technologists and other stakeholders.
- **Project chartering** – assesses whether a business justification exists for the project and whether it fits the organisation's overall goals.
- **Test-driven management** – establishes measurable "destinations" and mechanisms to determine whether they have been reached.
- **Retrospectives** – a specialised technical review after each software increment, examining issues, events and lessons learned.
- **Continuous learning** – team members are encouraged (and given incentives) to learn new methods and techniques.

IXP also modifies a number of existing XP practices (for example, story-driven development, domain-driven design, pairing and iterative usability).

The XP Debate

Critics of XP raise several concerns: **requirements volatility** (the customer is an active member of the team, and changes can be extensive); **conflicting customer needs** (many projects have multiple customers with different needs); **requirements expressed informally** (user stories and acceptance tests are the only explicit manifestation of requirements, so they may be incomplete); and **lack of formal design** (XP de-emphasises the need for architectural design, and for complex systems this may be risky). Supporters respond that XP is meant to be adopted with discipline and that many of these concerns apply equally to other methods.

3.5 Other Agile Process Models

The history of software engineering is littered with dozens of obsolete process descriptions and methodologies. Although each of the agile models shares a common philosophy, each emphasises different things. The most commonly used are summarised below.

(a) Adaptive Software Development (ASD)

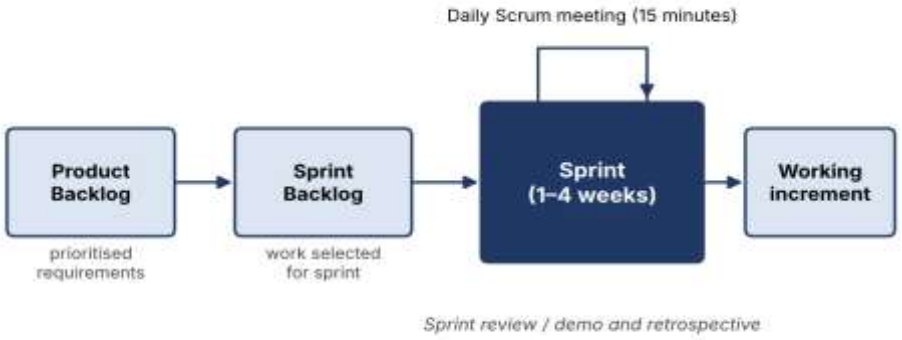
Proposed by **Jim Highsmith** as a technique for building complex software and systems. ASD focuses on human collaboration and team self-organisation. It embodies an **adaptive cycle** with three overlapping phases:

- **Speculation** – the project is initiated and adaptive cycle planning is conducted, using the customer's mission statement, project constraints and basic requirements to define the set of release cycles (software increments).
- **Collaboration** – motivated people use collaboration to multiply their talent and creative output beyond absolute number. It involves communication, teamwork, individual creativity and mutual trust.
- **Learning** – as team members begin to develop components, the emphasis is on learning, through **focus groups, technical reviews and project postmortems**. ASD teams learn in three ways.

(b) Scrum

Scrum was developed by **Jeff Sutherland** and his team in the early 1990s; further development was done by Schwaber and Beedle. Its principles are consistent with the agile manifesto and guide development activities within a process that incorporates the framework activities: **requirements, analysis, design, evolution and delivery**. Within each framework activity, work tasks occur within a process pattern called a **sprint**.

Fig. 3 Scrum process flow



- **Backlog** – a prioritised list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time (this is how changes are introduced). The product manager assesses the backlog and updates priorities as required.
- **Sprints** – consist of work units that are required to achieve a requirement defined in the backlog and that must fit into a predefined **time-box** (typically 30 days or less). Changes (e.g. backlog work items) are not introduced during the sprint, so the sprint allows team members to work in a short-term but stable environment.
- **Scrum meetings** – short (typically **15-minute**) meetings held daily by the Scrum team. A team leader, called the **Scrum master**, leads the meeting and assesses the responses from each person. Three key questions are asked of all team members: (1) What did you do since the last team

meeting? (2) What obstacles are you encountering? (3) What do you plan to accomplish by the next team meeting?

- **Demos** – deliver the software increment to the customer so that the functionality that has been implemented can be demonstrated and evaluated by the customer. The demo may not contain all planned functionality, but rather those functions that can be delivered within the time-box established.

(c) Dynamic Systems Development Method (DSDM)

DSDM is an agile software development approach that provides a framework for building and maintaining systems which meet tight time constraints through the use of **incremental prototyping** in a controlled project environment. It is a philosophy borrowed from a modified version of the **Pareto principle** – **80 percent of an application can be delivered in 20 percent of the time** it would take to deliver the complete (100 percent) application. DSDM is an iterative software process in which each iteration follows the 80 percent rule: only enough work is required for each increment to facilitate movement to the next increment; the remaining detail can be completed later when more business requirements are known or changes have been requested and accommodated.

The DSDM life cycle defines three different iterative cycles, preceded by two additional life-cycle activities:

- **Feasibility study** – establishes the basic business requirements and constraints and assesses whether the application is a viable candidate for DSDM.
- **Business study** – establishes the functional and information requirements that will allow the application to provide business value; also defines the basic application architecture and identifies the maintainability requirements.
- **Functional model iteration** – produces a set of incremental prototypes that demonstrate functionality for the customer. All prototypes are intended to evolve into the deliverable application; the intent is to gather additional requirements by eliciting feedback from users as they exercise the prototype.
- **Design and build iteration** – revisits prototypes built during the functional model iteration to ensure that each has been engineered in a manner that will enable it to provide operational business value for end users. This may occur concurrently with the functional model iteration.
- **Implementation** – places the latest software increment (an "operationalised" prototype) into the operational environment. The increment may not be 100 percent complete; changes may be requested as it is placed into operation, and development work continues by returning to the functional model iteration.

(d) Crystal

Developed by **Alistair Cockburn and Jim Highsmith**. The goal is to achieve a maneuverable process – "a resource-limited, cooperative game of invention and communication, with a primary goal of delivering useful, working software and a secondary goal of setting up for the next game." To achieve this, Cockburn and Highsmith defined a **family of agile methods** (colour-coded such as Crystal Clear, Crystal Yellow, Crystal Orange) sharing core characteristics but with unique roles, process patterns, work products and practices, chosen according to the project's size and criticality. The Crystal family recognises that different projects require different degrees of ceremony and rigour.

(e) Feature Driven Development (FDD)

Originally conceived by **Peter Coad** and his colleagues as a practical process model for object-oriented software engineering; Stephen Palmer and John Felsing extended and improved the work. FDD adopts a philosophy that (1) emphasises collaboration among people on an FDD team; (2) manages problem and project complexity using feature-based decomposition followed by integration of software increments; and (3) communicates technical detail using verbal, graphical and text-based means. FDD emphasises software quality assurance activities.

- A **feature** is a client-valued function that can be implemented in two weeks or less. Features are small, which makes them easier to describe, review, design and test.
- A feature is expressed in the form: **<action> the <result> <by | for | of | to> a(n) <object>** – for example, "Add the product to a shopping cart".
- The FDD approach defines five **collaborative framework activities (processes)**: develop an overall model; build a features list; plan by feature; design by feature; build by feature.

(f) Lean Software Development (LSD)

Lean Software Development adapts the principles of **lean manufacturing** to the software world. The lean principles that inspire the LSD process can be summarised as: **eliminate waste, build quality in, create knowledge, defer commitment, deliver fast, respect people, and optimise the whole.** Each principle suggests a set of agile practices; for example, "eliminate waste" means avoiding unnecessary features and functions, removing extra process steps and delays, and so on.

(g) Agile Modeling (AM)

Scott Ambler's **Agile Modeling** is a practice-based methodology for modelling and documentation of software-based systems. It is a collection of values, principles and practices that can be applied on a software development project in an effective and lightweight manner. Its principles include: **model with a purpose; use multiple models; travel light; content is more important than representation; know the models and the tools you use to create them; adapt locally.**

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

(h) Agile Unified Process (AUP)

AUP adopts a **"serial in the large" and "iterative in the small"** philosophy for building computer-based systems. By adopting the classic UP phased activities – inception, elaboration, construction and transition – AUP provides a serial overlay (a linear sequence of software engineering activities) that enables a team to visualise the overall process flow. Within each activity, the team iterates to achieve agility and to deliver meaningful software increments as rapidly as possible. Each AUP iteration addresses: modelling, implementation, testing, deployment, configuration and project management, and environment management.

Comparison at a glance

Model	Proposed by	Distinguishing idea
XP	Kent Beck	User stories, pair programming, test-first coding, refactoring, continuous integration
ASD	Jim Highsmith	Speculation → collaboration → learning adaptive cycle
Scrum	Jeff Sutherland (Schwaber & Beedle)	Backlog, time-boxed sprints, daily 15-minute Scrum meeting, demos
DSDM	DSDM Consortium	80/20 (Pareto) rule; time-boxed incremental prototyping
Crystal	Cockburn & Highsmith	Family of methods tuned to team size and project criticality
FDD	Peter Coad (Palmer & Felsing)	Client-valued features delivered in two weeks or less
LSD	Poppendieck (lean thinking)	Eliminate waste, defer commitment, deliver fast
AM	Scott Ambler	Lightweight modelling and documentation
AUP	Scott Ambler	Serial in the large, iterative in the small (UP phases)

3.6 A Tool Set for the Agile Process

Some agile developers resist the notion that automated software tools are an essential part of the agile process; they emphasise people and interaction over tools. Others argue that tools can make an agile team more effective, provided the tools themselves do not become a burden. In the agile context, the tool set is therefore **"social" and lightweight** – it supports collaboration, communication and quick feedback rather than prescribing a rigid method. Typical categories of tools used by agile teams are:

Category	Purpose and examples
Collaborative and communication tools	Support rapid, frequent communication among distributed team members and customers – chat/messaging, video conferencing, shared whiteboards, wikis and shared repositories for lightweight documentation.
Project management and tracking tools	Maintain the backlog, user stories, sprints and task boards (for example Kanban/Scrum boards in tools such as Jira or Trello), and show progress through burndown charts and velocity.
Version control and configuration management	Allow many developers to work on the same code base safely; essential to continuous integration (for example Git).
Build and test automation tools	Automated builds, continuous-integration servers and unit-test frameworks give the instant feedback that XP-style test-first development depends on.
Modelling and design aids	Lightweight tools for sketching CRC cards, simple UML and prototypes; the emphasis is on "just enough" modelling.

The common thread is that any tool adopted must reinforce the agile values – individuals and interactions, working software, customer collaboration and response to change.

PART B: PRINCIPLES THAT GUIDE PRACTICE (Chapter 4)

Software engineering practice is a broad array of **principles, concepts, methods and tools** that you must consider as software is planned and developed. Practice encompasses the technical activities that produce all work products that are defined by the software process model chosen. This chapter introduces the principles that serve as a foundation for good practice.

4.1 Software Engineering Knowledge

In an editorial in IEEE Software, Steve McConnell made the point that the knowledge of specific technologies has a very short **"half-life"** – roughly three years – meaning that half of what you know about a particular technology will be obsolete in that time. By contrast, knowledge of **fundamental principles** has a long half-life that can be measured in decades. Software engineers therefore need to invest in principles: they provide a foundation from which you can learn new methods, techniques and tools as they appear. Principles also give the engineer a way to evaluate whether a given technology or method is sound.

4.2 Core Principles

The following core principles are drawn at two levels – principles that guide the **process** as a whole, and principles that guide **practice** at the level of individual engineers.

Principles that guide process

1. **Be agile.** Whether the process model is prescriptive or agile, the basic tenets of agile development should govern your approach – keep technical approach and work products as simple as possible.
2. **Focus on quality at every step.** The exit condition for every process activity, action and task should focus on the quality of the work product produced.
3. **Be ready to adapt.** Process is not a religious experience; adapt your approach to the problem and to the constraints.
4. **Build an effective team.** Software engineering process and practice are important, but the bottom line is people – build a self-organising team with mutual trust and respect.
5. **Establish mechanisms for communication and coordination.** Projects fail because important information falls into the cracks and/or stakeholders fail to coordinate their efforts.
6. **Manage change.** Formal or informal mechanisms must be established to manage the way changes are requested, assessed, approved and implemented.
7. **Assess risk.** Lots of things can go wrong; contingency plans should be established.
8. **Create work products that provide value for others.** Create only those work products that provide value for other process activities, actions or tasks.

Principles that guide practice

1. **Divide and conquer.** Stated in a more technical manner, analysis and design should always emphasise **separation of concerns (SoC)**. A large problem is easier to solve if it is subdivided into a collection of elements, each of which delivers distinct functionality.
2. **Understand the use of abstraction.** An abstraction is a simplification of some complex element of a system used to communicate meaning in a single phrase. Abstraction is used to eliminate unnecessary detail so the problem can be reasoned about at the right level.
3. **Strive for consistency.** Whether it is creating a requirements model, developing a software design, generating source code or creating test cases, the principle of consistency suggests that a familiar context makes software easier to use.
4. **Focus on the transfer of information.** Software is about information transfer – from a database to an end user, from a legacy system to a WebApp, from an end user into a graphical user interface, from an operating system to an application, from one software component to another. Pay special attention to the analysis, design, construction and testing of interfaces.
5. **Build software that exhibits effective modularity.** Separation of concerns establishes a philosophy for software; **modularity** provides a mechanism for realising the philosophy. Any complex system can be divided into modules (components), but good software engineering practice

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

demands more – modularity must be effective, each module focused exclusively on one well-constrained aspect of the system.

6. **Look for patterns.** The goal of patterns within the software community is to create a body of literature to help software developers resolve recurring problems encountered throughout all of software development.
7. **When possible, represent the problem and its solution from a number of different perspectives.** When a problem and its solution are examined from different perspectives, it is more likely that greater insight will be achieved and that errors and omissions will be uncovered.
8. **Remember that someone will maintain the software.** Over the long term, software will be corrected as defects are uncovered, adapted as its environment changes, and enhanced as stakeholders request more capabilities. Maintainability should be designed in from the start.

Hooker's Seven Principles of Software Engineering

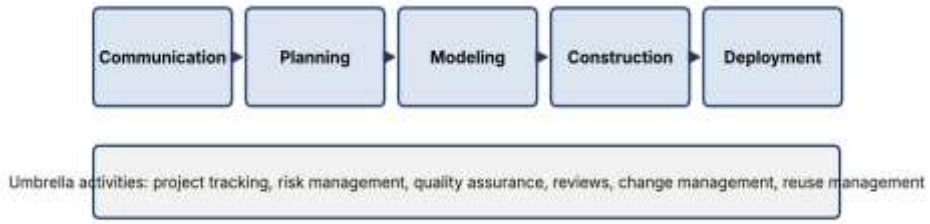
David Hooker proposed seven principles that focus on software engineering practice as a whole. Because of their general importance, they are reproduced here in short form:

#	Principle	Essence
1	**The Reason It All Exists**	A software system exists to provide value to its users. All decisions should be made with this in mind – if a feature does not add real value, do not build it.
2	**KISS (Keep It Simple, Stupid)**	All design should be as simple as possible, but no simpler. Simple does not mean "quick and dirty"; it often takes much thought and work over multiple iterations.
3	**Maintain the Vision**	A clear vision is essential to the success of a software project. Compromising the architectural vision weakens even the best-designed system.
4	**What You Produce, Others Will Consume**	Always specify, design and implement knowing someone else will have to understand and maintain what you do. Make the code readable and documentation clear.
5	**Be Open to the Future**	Never design yourself into a corner. Systems with a long lifetime have more value; build for change and be ready to adapt.
6	**Plan Ahead for Reuse**	Reuse saves time and effort. Planning for it at the start reduces the cost and increases the value of both the reusable components and the systems that use them.
7	**Think!**	Placing clear, complete thought before action almost always produces better results. Thinking leads to learning and to better work.

4.3 Principles that Guide Each Framework Activity

The generic framework activities – **communication, planning, modeling, construction and deployment** – each have their own guiding principles. These principles are the "how-to" that stay valid whichever process model is in use.

Fig. 4 Generic framework activities that carry the principles



A. Communication Principles

Before customer requirements can be analysed, modelled or specified they must be gathered through the communication activity. Communication is difficult and, when done badly, is a major cause of project failure. Ten principles apply:

1. **Listen.** Try to focus on the speaker's words rather than formulating your response to those words. Ask for clarification if something is unclear, but avoid constant interruptions.
2. **Prepare before you communicate.** Spend the time to understand the problem before you meet with others. If necessary, do some research to understand business domain jargon.
3. **Someone should facilitate the activity.** Every communication meeting should have a leader (a facilitator) to keep the conversation moving in a productive direction, to mediate any conflict, and to ensure that other principles are followed.
4. **Face-to-face communication is best.** But it usually works better when some other representation of the relevant information is present – for example, a drawing or a "strawman" document.
5. **Take notes and document decisions.** Things have a way of falling into the cracks. Someone participating in the communication should serve as a "recorder".
6. **Strive for collaboration.** Collaboration and consensus occur when the collective knowledge of members of the team is used to describe product or system functions or features.
7. **Stay focused; modularise your discussion.** The more people involved in any communication, the more likely that discussion will bounce from one topic to the next. The facilitator should keep the conversation modular.

8. **If something is unclear, draw a picture.** Verbal communication goes only so far. A sketch or drawing can often provide clarity when words fail to do the job.
9. **(a) Once you agree to something, move on. (b) If you can't agree to something, move on. (c) If a feature or function is unclear and cannot be clarified at the moment, move on.** Communication, like any software engineering activity, takes time. Rather than iterating endlessly, the people who participate should recognise that many topics require discussion and that "moving on" is sometimes the best way to achieve communication agility.
10. **Negotiation is not a contest or a game.** It works best when both parties win. There are many instances in which you and other stakeholders must negotiate functions and features, priorities and delivery dates; the team must work collaboratively with stakeholders.

B. Planning Principles

The planning activity encompasses a set of management and technical practices that enable the software team to define a roadmap as it travels toward its strategic goal and tactical objectives. Ten principles guide planning:

1. **Understand the scope of the project.** It is impossible to use a roadmap if you do not know where you are going. Scope provides the software team with a destination.
2. **Involve the customer in the planning activity.** The customer defines priorities and establishes project constraints. To accommodate these realities, software engineers must often negotiate order of delivery, timelines and other project-related issues.
3. **Recognise that planning is iterative.** A project plan is never engraved in stone. As work begins, it is very likely that things will change; as a consequence the plan must be adjusted.
4. **Estimate based on what you know.** The intent of estimation is to provide an indication of effort, cost and task duration based on the team's current understanding of the work to be done. If information is vague or unreliable, estimates will be equally unreliable.
5. **Consider risk as you define the plan.** If the team has defined risks that have high impact and high probability, contingency planning is necessary.
6. **Be realistic.** People do not work 100 percent of every day. Noise always enters any human communication. Omissions and ambiguity are facts of life. Change will occur. Even the best software engineers make mistakes. These and other realities should be considered as a project plan is established.
7. **Adjust granularity as you define the plan.** Granularity refers to the level of detail that is introduced as a project plan is developed. A "fine-granularity" plan provides significant work task detail planned over relatively short time increments; a "coarse-granularity" plan provides broader work tasks planned over longer time periods.

8. **Define how you intend to ensure quality.** The plan should identify how the software team intends to ensure quality – for example, formal technical reviews and a pair-programming approach.
9. **Describe how you intend to accommodate change.** Even the best planning can be obviated by uncontrolled change. The team should identify how changes are to be accommodated as software engineering work proceeds.
10. **Track the plan frequently and make adjustments as required.** Software projects fall behind schedule one day at a time. Therefore, it makes sense to track progress on a daily basis, looking for problem areas and situations in which scheduled work does not conform to actual work conducted.

Barry Boehm's W5HH Principle

Boehm suggests an approach that addresses project objectives, milestones and schedules, responsibilities, management and technical approaches, and required resources. It is summarised as a series of questions:

Why is the system being developed? **What** will be done? **When** will it be accomplished?

Who is responsible for each function? **Where** are they located (organisationally)?

How will the job be done technically and managerially? **How much** of each resource is needed?

C. Modeling Principles

We create models to gain a better understanding of the actual entity to be built. Models are created in two classes: **requirements models** (analysis) and **design models**.

Requirements (analysis) modeling principles – a set of operational principles that guide requirements modelling:

1. The **information domain** of a problem must be represented and understood.
2. The **functions** that the software performs must be defined.
3. The **behaviour** of the software (as a consequence of external events) must be represented.
4. The models that depict information, function and behaviour must be **partitioned** in a manner that uncovers detail in a layered (or hierarchical) fashion.
5. The analysis task should move from **essential information toward implementation detail**.

Design modeling principles – the design model created for software provides a variety of different views of the system. Principles proposed by Davis and others include:

1. **Design should be traceable to the requirements model.**
2. **Always consider the architecture of the system to be built.**

3. **Design of data is as important as design of processing functions.**
4. **Interfaces (both internal and external) must be designed with care.**
5. **User interface design should be tuned to the needs of the end user** and should stress ease of use.
6. **Component-level design should be functionally independent** (high cohesion).
7. **Components should be loosely coupled** to one another and to the external environment.
8. **Design representations (models) should be easily understandable.**
9. **The design should be developed iteratively.** With each iteration, the designer should strive for greater simplicity.

Agile modeling principles – where an agile approach is used, the following principles guide the modelling work:

1. The primary goal of the software team is to build software, not create models.
2. Travel light – do not create more models than you need.
3. Strive to produce the simplest model that will describe the problem or the software.
4. Build models in a way that makes them amenable to change.
5. Be able to state an explicit purpose for each model that is created.
6. Adapt the models you develop to the system at hand.
7. Try to build useful models, but forget about building perfect models.
8. Do not become dogmatic about model syntax. Successful communication is key.
9. If your instincts tell you a model is not right even though it seems okay on paper, you probably have reason to be concerned.
10. Get feedback as soon as you can.

D. Construction Principles

The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end user. The principles are grouped as follows.

Preparation principles (before you write a single line of code):

1. Understand the problem you are trying to solve.
2. Understand basic design principles and concepts.
3. Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.
4. Select a programming environment that provides tools that will make your work easier.
5. Create a set of unit tests that will be applied once the component you code is completed.

Coding principles (as you begin writing code):

1. Constrain your algorithms by following structured programming practice.
2. Consider the use of pair programming.
3. Select data structures that will meet the needs of the design.
4. Understand the software architecture and create interfaces that are consistent with it.
5. Keep conditional logic as simple as possible.
6. Create nested loops in a way that makes them easily testable.
7. Select meaningful variable names and follow other local coding standards.
8. Write code that is self-documenting.
9. Create a visual layout (for example, indentation and blank lines) that aids understanding.

Validation principles (after you have completed your first coding pass):

1. Conduct a code walkthrough when appropriate.
2. Perform unit tests and correct errors you have uncovered.
3. Refactor the code.

Testing principles – Davis suggests a set of testing principles; Pressman adds others:

1. **All tests should be traceable to customer requirements.**
2. **Tests should be planned long before testing begins.**
3. The **Pareto principle** applies to software testing – 80 percent of all errors uncovered during testing will likely be traceable to 20 percent of all program components.
4. **Testing should begin "in the small" and progress toward testing "in the large"** (components first, then integrated clusters, then the whole system).
5. **Exhaustive testing is not possible.**

E. Deployment Principles

The deployment activity encompasses three actions: **delivery, support and feedback**. Because modern software process models are evolutionary or incremental, deployment happens not once but many times as each software increment is presented to the customer. Key principles are:

1. **Customer expectations for the software must be managed.** Too often, the customer expects more than the team has promised to deliver, and disappointment occurs immediately.
2. **A complete delivery package should be assembled and tested.**
3. **A support regime must be established before the software is delivered.** An end user expects responsiveness and accurate information when a question or problem arises.

4. **Appropriate instructional materials must be provided to end users.**
5. **Buggy software should be fixed first, delivered later.** Under time pressure, some software organisations deliver low-quality increments with a warning that bugs will be fixed in the next release – a mistake.

IMPORTANT QUESTIONS – MODULE 3 (VTU Pattern)

The following questions follow the pattern of the VTU question paper (each module carries two full questions of 20 marks, with sub-parts). Use them for practice; they are not a record of exact previous-year papers.

Agile Development

1. What is agility? Explain agility and the cost of change with a neat diagram. (8)
2. List and explain the twelve principles of agility given by the Agile Alliance. (10)
3. What is an agile process? Explain the three key assumptions and the human factors that agile team members should possess. (10)
4. Explain the Extreme Programming (XP) process with a neat diagram. Mention the XP values. (12)
5. Explain the following XP activities: (a) Planning (b) Design (c) Coding (d) Testing. (4 × 3)
6. Write short notes on Industrial XP and discuss the XP debate. (8)
7. Explain Scrum with a diagram. Describe backlog, sprints, Scrum meetings and demos. (10)
8. Explain the DSDM life cycle and the 80/20 rule. (8)
9. Write short notes on any three: ASD, Crystal, FDD, Lean Software Development, Agile Modeling, AUP. (3 × 4)
10. Discuss the tool set that supports an agile process. (5)

Principles that Guide Practice

1. Explain the core principles that guide process and the principles that guide practice. (10)
2. List and explain David Hooker's seven principles of software engineering practice. (10)
3. Explain the communication principles. (8)
4. Explain the planning principles and Boehm's W5HH principle. (10)
5. Explain the modeling principles – requirements (analysis) modeling and design modeling. (10)
6. Explain the construction principles – preparation, coding, validation and testing principles. (10)
7. Explain the deployment principles. (5)
8. What is meant by the half-life of software engineering knowledge? Why are principles more important than technologies? (5)

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106